

# 软工二复习重点

## 总口诀

内部暴露 -> 信息隐藏、迪米特法则

职责混乱 -> SRP、高内聚差

直接依赖实现 -> DIP

新增规则总改老代码 -> OCP

接口太大 -> ISP

替换实现出错 -> LSP

包互相依赖 -> ADP

重复逻辑 -> DRY

设计太复杂 -> KISS

提前做没用功能 -> YAGNI

## 信息隐藏

**核心：** 模块只暴露必要接口，隐藏内部数据结构和实现细节。

**题干信号：**

- 直接访问内部对象
- 暴露内部数据结构
- 字段变化影响多个包

**答题句：**

违反信息隐藏原则，因为内部数据结构和变化点没有被封装，其他模块可以直接依赖内部实现。

## SRP：单一职责原则

**核心：** 一个类或包只负责一类职责，只有一个主要变化理由。

**题干信号：**

- 职责边界不清
- 一个类或包做太多事
- 业务规则、通知、资源状态、用户权限混在一起

**答题句：**

违反 SRP，因为包和类承担多个职责，存在多个变化理由。

## OCP：开闭原则

**核心：** 对扩展开放，对修改关闭。

**题干信号：**

- 新增规则要修改原有主流程
- 变化点没有封装
- 每加一种功能都要改旧代码

**答题句：**

违反 OCP，因为新增业务规则时需要修改已有主流程，而不是通过新增扩展类完成。

**例子：**

预约冲突检测规则可以抽成 `ConflictCheckStrategy`。新增时间段冲突、容量冲突、权限冲突、设备状态冲突时，新增策略实现类。

## LSP：里氏替换原则

**核心：** 子类对象应该可以替换父类对象，并且程序行为不被破坏。

**题干信号：**

- 子类替换父类后行为不一致
- 实现类不遵守接口语义
- 换一个实现后业务出错

答题句：

违反 LSP，因为子类或实现类不能在不破坏原有行为的情况下替换父类或接口。

例子：

ReservationRepository 的内存实现和数据库实现都应该遵守 save 的语义。

按资源和时间段查询已有预约的方法，也应该在不同实现中保持相同语义。

## ISP：接口隔离原则

核心： 接口要小而专一，不要让调用方依赖自己不用的方法。

题干信号：

- 接口太大
- 一个接口塞很多职责
- 调用方被迫依赖不需要的方法

答题句：

违反 ISP，因为接口职责过大，调用方依赖了自己不需要的的方法。

例子：

不要把预约、工单、通知、用户、资源都塞进 CampusService。应该拆成 ReservationService、WorkOrderService、NotificationGateway、UserService、ResourceQueryService。

## DIP：依赖倒置原则

核心： 高层业务不直接依赖低层实现，双方都依赖抽象接口。

题干信号：

- 高层业务直接依赖具体实现
- 直接 new 低层对象
- 直接访问低层内部对象

答题句：

违反 DIP，因为高层业务直接依赖低层实现对象，而不是依赖稳定接口。

正确方向：

ReservationService -> ReservationRepository 接口

MysqlReservationRepository -> 实现 ReservationRepository

错误方向:

ReservationService -> MysqlReservationRepository

## Law of Demeter: 迪米特法则

核心: 一个对象只和直接相关的对象通信, 不要穿透别人内部结构。

题干信号:

- 跨对象访问内部结构
- 链式访问过多
- 知道太多别的对象细节

答题句:

违反 Law of Demeter, 因为对象跨越边界访问其他对象的内部结构, 知道了过多实现细节。

坏味道:

a.getB().getC().getD()

## ADP: 无环依赖原则

核心: 包之间的依赖关系不能形成环。

题干信号:

- 包互相依赖
- 循环依赖
- A 依赖 B, B 又依赖 A

答题句:

违反 ADP, 因为包依赖关系中存在环, 修改一个包会牵连其他包。

错误例子:

reservation -> resource

resource -> reservation

## DRY：不要重复

**核心：** 同一知识或规则只保留一份。

**题干信号：**

- 同一逻辑多处复制
- 规则散落在多个地方
- 改一处还要同步改多处

**答题句：**

违反 DRY，因为相同规则在多个位置重复出现，导致修改成本和不一致风险增加。

## KISS：保持简单

**核心：** 能简单解决，就不要设计得过度复杂。

**题干信号：**

- 设计过度
- 简单问题复杂化
- 结构很绕但收益不明显

**答题句：**

违反 KISS，因为设计复杂度超过当前问题需要，增加了理解和维护成本。

## YAGNI：不要提前设计用不到的东西

**核心：** 当前没有明确需求的功能，不要提前实现。

**题干信号：**

- 为了未来功能提前写大量代码
- 当前需求用不到
- 过早抽象

**答题句：**

违反 YAGNI，因为实现了当前需求并不需要的功能或抽象。

## 高内聚、低耦合

**核心：** 高内聚是一个模块内部的内容围绕同一职责组织。低耦合是模块之间尽量少知道彼此细节。

**题干信号：**

- 包之间互相直接访问内部对象 -> 低耦合被破坏
- 多个职责混在不同包中 -> 高内聚被破坏
- 共享可变全局配置 -> 公共耦合

**第六题可用答案：**

从耦合角度看，各包互相直接访问内部对象，导致高访问耦合和循环依赖；多个包共享同一个可变全局配置对象，形成公共耦合。

从内聚角度看，预约、资源、工单、通知和用户权限等职责混在不同包中，职责边界不清，导致模块内聚性下降。

## 人机交互 HCI 原则

**HCI 含义：** HCI 是 Human-Computer Interaction，即人机交互。HCI 原则用于指导界面和交互设计，使用户看得懂、学得快、少出错，出错后能恢复，操作效率高，并保持体验一致。

**总口诀：**

状态要看见，错误要预防，选项要看得见，界面要一致，用户能撤回，按钮要好点，页面要简洁，出错能恢复。

## 系统状态可见

**核心：** 系统要让用户知道当前发生了什么。

**题干信号：**

- 提交中
- 审核中
- 预约成功
- 冲突失败
- 处理结果通知

**答题句：**

系统状态可见：提交中、审核中、预约成功、冲突失败必须明确显示，避免用户提交后不知道系统是否处理。

## 错误预防

**核心：** 尽量在错误发生前阻止错误。

**题干信号：**

- 非法时间段
- 已满资源
- 权限不足
- 提交前校验

**答题句：**

错误预防：非法时间段、已满资源和权限不足应在提交前提示，减少用户提交后失败的次数。

## 识别优于回忆

**核心：** 让用户直接看到可选内容，而不是要求用户凭记忆输入。

**题干信号：**

- 资源日历
- 时间段选择
- 可预约时段直接显示
- 资源名称、地点、容量直接展示

**答题句：**

识别优于回忆：资源日历直接显示可预约时段和资源信息，减少用户记忆资源编号和空闲时间。

## 一致性和标准

**核心：** 相同功能在不同页面中应保持术语、颜色、按钮位置和交互方式一致。

**题干信号：**

- 多个页面术语不统一
- 按钮位置变化大
- 成功、失败提示样式不一致

**答题句：**

一致性和标准：按钮位置、颜色、术语在预约、工单和报表页面保持一致，降低用户学习成本。

## 用户控制和自由

**核心：** 用户应该能撤销、返回、修改、取消和重新提交。

**题干信号：**

- 取消预约

- 修改时间段
- 返回列表
- 重新提交

答题句：

用户控制和自由：提供取消预约、修改时间段、返回列表和重新提交入口，避免用户选错后无法恢复。

## Fitts 定律

核心： 常用目标越大、离操作区域越近，越容易点击。

题干信号：

- 提交按钮
- 高频按钮
- 按钮尺寸
- 按钮距离主要操作区域

答题句：

Fitts 定律：提交预约等高频按钮应靠近主要操作区域，按钮尺寸足够大，方便用户快速点击。

## 美学和极简

核心： 页面应突出当前任务需要的信息，不堆积无关内容。

题干信号：

- 资源日历
- 可选时间段
- 冲突提示
- 历史记录和高级筛选折叠

答题句：

美学和极简：默认显示资源日历、可选时间段、冲突提示和提交按钮等关键任务信息，历史记录和高级筛选可以折叠。

## 帮助用户恢复错误

**核心：** 出错时不只提示失败，还要说明原因并给出恢复路径。

**题干信号：**

- 冲突提示
- 失败原因
- 可选时间段
- 重新选择入口

**答题句：**

帮助用户恢复错误：冲突提示应给出失败原因和可选时间段，而不是只显示预约失败。

## 第十题可用答案

1. 系统状态可见：提交中、审核中、预约成功、冲突失败必须明确显示。
2. 错误预防：非法时间段、已满资源和权限不足应在提交前提示。
3. 识别优于回忆：资源日历直接显示可预约时段，减少用户记忆资源编号。
4. 一致性和标准：按钮位置、颜色、术语在预约、工单和报表页面保持一致。
5. 用户控制和自由：提供取消预约、修改时间段、返回列表和重新提交入口。
6. Fitts 定律：提交按钮靠近主要操作区域，按钮尺寸足够大。
7. 美学和极简：默认显示关键任务信息，历史记录和高级筛选折叠。
8. 帮助用户恢复错误：冲突提示应给出可选时间段，而不是只显示失败。

## 软件开发生命周期与质量保障阶段

### 完整软件开发生命周期：

1. 需求工程 / 软件需求
2. 软件设计
3. 软件构造 / 软件实现
4. 软件测试
5. 软件交付
6. 软件维护

### 英文对应：

Software Requirement

Software Design

Software Construction

Software Testing

Software Deliver

Software Maintenance

### 卷子里的质量保障阶段：

需求阶段

体系结构阶段

详细设计阶段

构造阶段

测试阶段

### 为什么卷子把软件设计拆开：

卷子第三问问的是质量保障活动覆盖哪些阶段，不是单纯列生命周期。

软件设计在质量保障中通常要拆成体系结构阶段和详细设计阶段，因为两者关注点不同、检查对象不同、质量活动也不同。

体系结构阶段关注系统整体结构、分层、模块划分、外部接口、技术方案和关键质量属性，所以要做架构评审，并提前考虑集成测试策略。

详细设计阶段关注类、接口、方法、模块内部职责、算法、数据结构和协作细节，所以要做设计评审、设计度量和集成测试准备。

### 关系记法：

完整生命周期：需求 -> 设计 -> 构造 -> 测试 -> 交付 -> 维护

质量保障答法：需求 -> 体系结构 -> 详细设计 -> 构造 -> 测试

其中：体系结构阶段 + 详细设计阶段 = 软件设计阶段

### **考试答题提醒：**

如果题目问软件生命周期，写需求、设计、构造、测试、交付、维护。

如果题目问质量保障活动覆盖哪些阶段，按卷子写需求阶段、体系结构阶段、详细设计阶段、构造阶段、测试阶段，并分别写对应的评审、度量、测试准备和测试执行活动。